CS 10: Operating Systems Laboratory – Assessment Task 10

Investigating Deadlocks

Name: Rodnee Glen A. Martin

Year & Section: 3 BSCS A

Simulator: YASMIN CPU-OS Simulator 7.5.50

Date Performed: October 23, 2026

1. Resource allocation graph for a four-process deadlock

Draw the resource allocation graph for a four process deadlock condition:

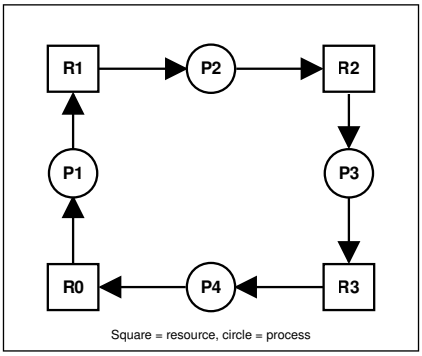


Figure 1. Resource allocation graph (an arrow from a resource to a process means "allocated to"; an arrow from a process to a resource means "requests").

Process	Holds (allocated first)	Then requests	Requested resource is held by
P1	R0	R1	P2
P2	R1	R2	P3
P3	R2	R3	P4
P4	R3	R0	P1

Each process holds one resource and waits for the resource held by the next process, so the arrows form one closed cycle: P1 → R1 → P2 → R2 → P3 → R3 → P4 → R0 → P1. Since there is only one instance of each resource, this cycle means deadlock. No process can continue, so none will ever release what it holds.

2. Creating the deadlock in the simulator

The values of X and Y were taken from the graph above (X = the resource held, Y = the resource requested):

```
program DeadlockP1
    resource(0, allocate)
    wait(3)
    resource(1, allocate)
    for n = 1 to 20
    next
end
```

```
program DeadlockP2
    resource(1, allocate)
    wait(3)
    resource(2, allocate)
    for n = 1 to 20
    next
end
```

```
program DeadlockP3
    resource(2, allocate)
    wait(3)
    resource(3, allocate)
    for n = 1 to 20
    next
end
```

```
program DeadlockP4
    resource(3, allocate)
    wait(3)
    resource(0, allocate)
    for n = 1 to 20
    next
end
```

The four programs were compiled and loaded in memory, and one process of each was created. Round Robin was selected, the speed was set to the fastest, and VIEW RESOURCES… was opened with Stay on top checked. Then START was clicked.

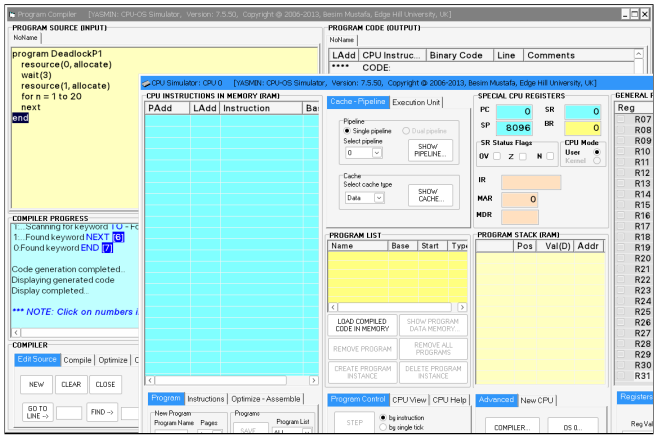


Figure 2. DeadlockP1 compiled in the Program Compiler (the other three were compiled the same way).

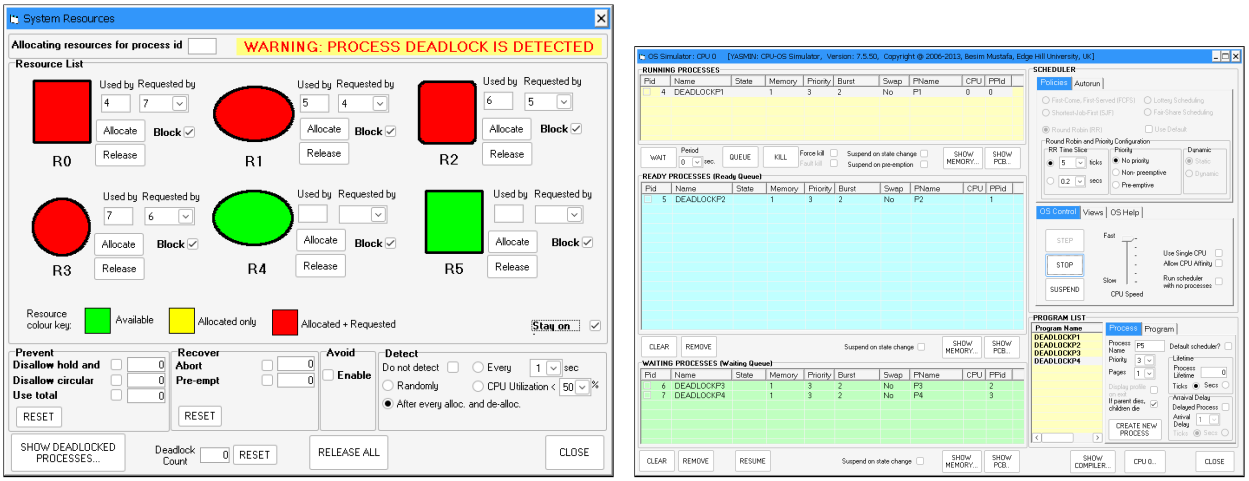


Figure 3. System Resources showing R0–R3 in red, and the deadlock warning (left); the OS simulator with all four processes stuck in the waiting queue (right).

m. Have you got a deadlock condition same as you constructed in (1)?

Yes. After a few seconds all four processes moved to the WAITING queue and stayed there, and the System Resources window showed “WARNING: PROCESS DEADLOCK IS DETECTED”. R0 to R3 turned red (allocated and requested): R0 used by P1 (PID 4) and requested by P4 (PID 7); R1 used by P2 (5) and requested by P1 (4); R2 used by P3 (6) and requested by P2 (5); R3 used by P4 (7) and requested by P3 (6).

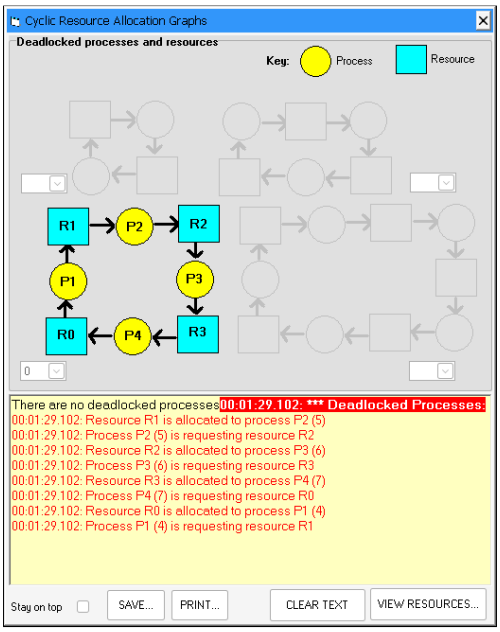


Figure 4. SHOW DEADLOCKED PROCESSES… window.

n. Does the highlighted resource allocation graph look like yours?

Yes. The highlighted graph is the same cycle as Figure 1: R1 → P2 → R2 → P3 → R3 → P4 → R0 → P1 → R1. The text below the graph lists the same thing: “Resource R1 is allocated to process P2 (5), Process P2 (5) is requesting resource R2 … Resource R0 is allocated to process P1 (4), Process P1 (4) is requesting resource R1.”

3. Two methods of recovering from the deadlock

Method 1 – releasing a resource. R0 was selected and its Release button was clicked.



Figure 5. Right after releasing R0: R0 is now used by P4 (PID 7) and turns yellow (allocated only).

c–d. What happened to the processes? Is the deadlock resolved? Explain why:

Yes, the deadlock was resolved. R0 was taken away from P1 and given straight away to P4, which was waiting for it. P4 moved to RUNNING, finished its loop and ended, which freed R3. P3 then got R3 and finished, which freed R2 for P2. P2 finished and freed R1 for P1, and P1 finished last. All four processes terminated. Releasing one resource broke the circular wait: one link in the cycle (R0 → P1) was removed, so P4 was no longer waiting, and the processes could finish one after another.

Method 2 – removing a process. The same deadlock was re-created. The OS was suspended (the simulator asks for this before REMOVE), P1 was selected in the WAITING PROCESSES frame, REMOVE was clicked, and the OS was resumed.

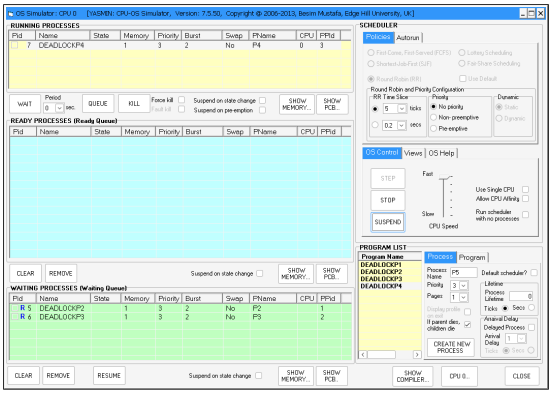


Figure 6. After removing P1: P4 is running, and P2 and P3 are still waiting.

h. Has this managed to resolve the deadlock? Explain why:

Yes. When P1 was removed, the OS took back the resource it held (R0). P4 got R0 and ran to the end, which freed R3 for P3. P3 then finished and freed R2 for P2, and P2 finished too. Removing (aborting) one process breaks the cycle because a process in the cycle disappears and its resources become free. The cost is that P1's work is lost and it never finished.

These two are methods of recovering from a deadlock after it happens: pre-empting a resource and aborting a process.

4. Two methods of preventing the deadlock

a–b. Disallow hold and wait.

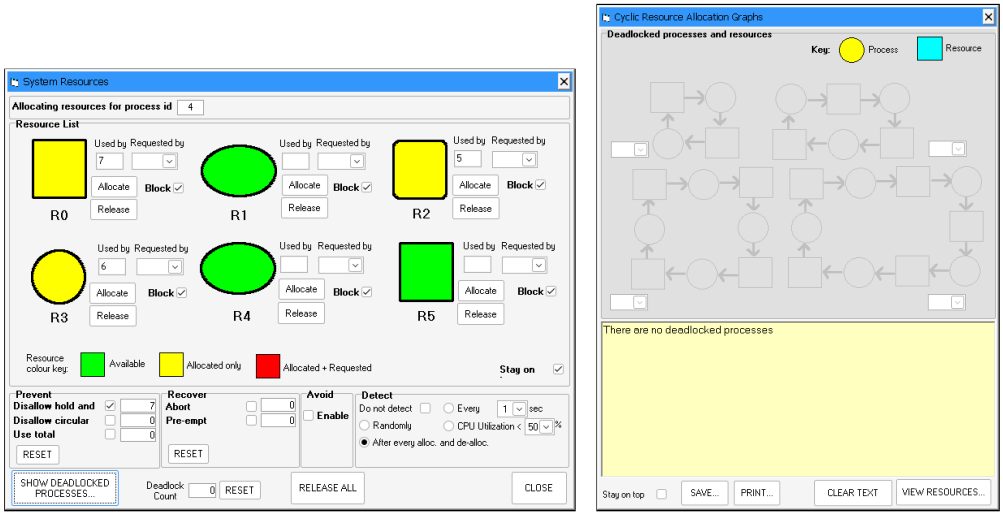


Figure 7. Disallow hold and wait checked: no red resources and a counter of 7 refusals (left); "There are no deadlocked processes" (right).

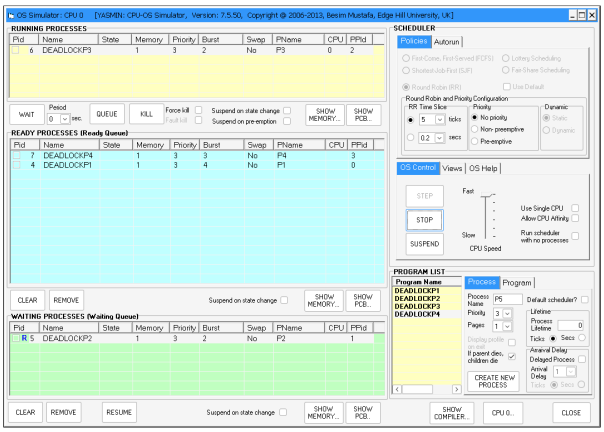


Figure 8. The processes keep going between running and ready instead of getting stuck in the waiting queue.

Have you been successful? What happened?

No, the deadlock could not be created. No resource ever turned red, and SHOW DEADLOCKED PROCESSES… said “There are no deadlocked processes”. The counter next to “Disallow hold and wait” went up to 7. Whenever a process that already held a resource requested another one that was busy, the OS did not let it hold one while waiting for the other: the request was refused and the held resource was taken back (for example, R0 was given to P4, PID 7). The processes went back to READY and tried again instead of waiting forever. After some retries all four processes finished.

c–d. Disallow circular wait (hold and wait unchecked).

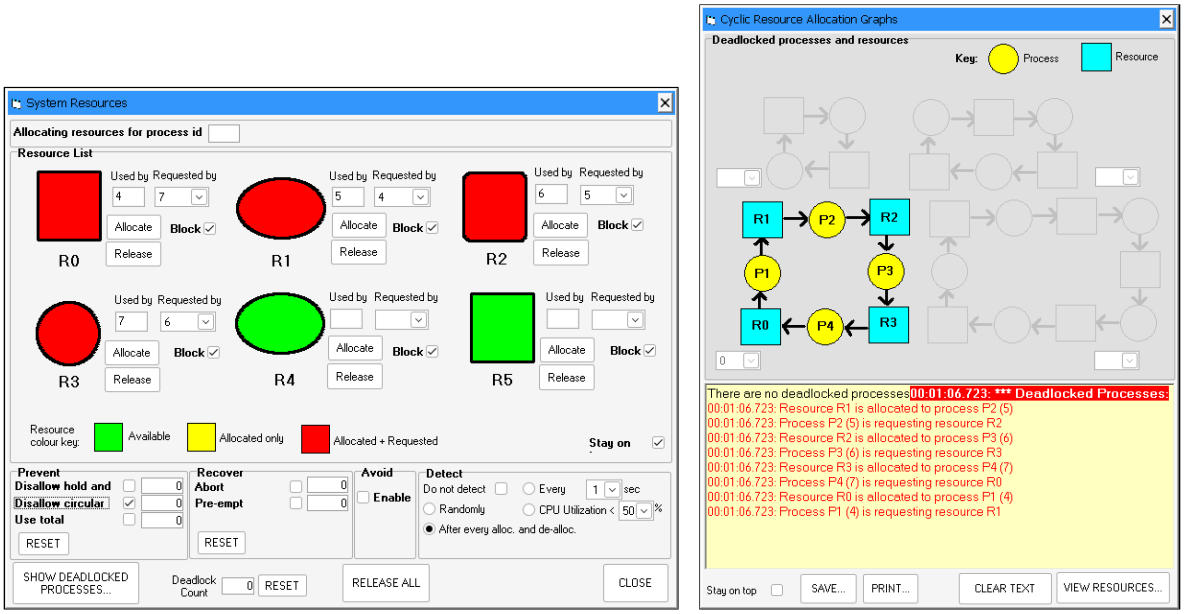


Figure 9. Disallow circular wait checked: the deadlock still happens, and its counter stays at 0.

Have you been successful? What happened?

Yes, the deadlock was still created. All four processes ended up in the WAITING queue, the window showed “PROCESS DEADLOCK IS DETECTED”, and SHOW DEADLOCKED PROCESSES… listed the same P1 → P2 → P3 → P4 cycle. The “Disallow circular wait” counter stayed at 0, so the option never refused a request. I repeated the test three times, after restarting the simulator and with a different detection setting, and got the same result each time. The idea of this method is to refuse any request

that would close a cycle in the graph (here, P4's request for R0). The simulator includes a "Circular waits are not allowed" message, but in this version that check did not happen for our programs, so this option did not prevent the deadlock.

5. Total ordering

Looking at your resource allocation graph can you see how this ordering can prevent a deadlock? Comment:

Yes. In the graph, P1, P2 and P3 already ask for resources in increasing order (R0 then R1, R1 then R2, R2 then R3). Only P4 breaks the rule: it holds R3 and then asks for R0, a lower number. That request is the arrow that closes the cycle. If every process must ask in increasing order, a cycle cannot form: every wait points from a lower-numbered resource to a higher one, and a chain that only goes up can never come back to where it started. So with total ordering, P4's request for R0 would not be allowed and the cycle is never completed.

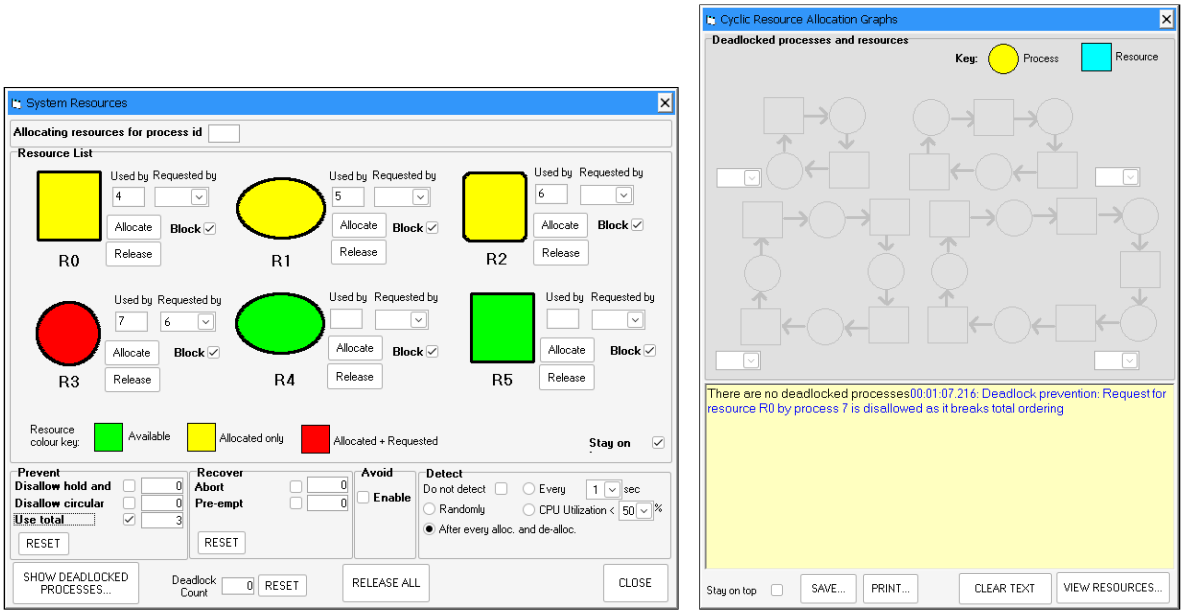


Figure 10. Use total ordering checked: counter of 3 refusals, and the message "Request for resource R0 by process 7 is disallowed as it breaks total ordering".

b. Have you been successful? What happened? Comment:

No, the deadlock was not created. The text window showed "Deadlock prevention: Request for resource R0 by process 7 is disallowed as it breaks total ordering", and the "Use total ordering" counter went up to 3. Exactly as predicted, P4 (PID 7) was the only process whose request was refused, because it holds R3 and asked for the lower-numbered R0. Since that request was not allowed, the cycle was never closed. P1, P2 and P3 could each get their two resources in turn, P4 retried later, and all four processes finished. This shows that total ordering prevents deadlock by removing the circular-wait condition, while still letting the processes run.